

Recurrent Neural Networks (RNNs) - Detailed Notes

Recurrent Neural Networks (RNNs) are a class of neural networks designed for processing sequential data. Unlike traditional feedforward networks, RNNs maintain a **hidden state** that captures information from previous time steps, making them suitable for time-series prediction, language modeling, and other sequential tasks.

A standard RNN follows the recurrence relation:

$$h_t = f(W_h h_{t-1} + W_x x_t + b), \quad (1)$$

where:

- h_t is the hidden state at time step t ,
- W_h and W_x are weight matrices,
- x_t is the current input,
- b is a bias term,
- $f(\cdot)$ is a nonlinearity (e.g., tanh or ReLU).

The output is given by:

$$y_t = g(W_y h_t + b_y), \quad (2)$$

where W_y maps the hidden state to the output space.

Memory Retention in RNNs

One of the key properties of RNNs is their ability to **capture memory** from previous time steps. To rigorously understand why RNNs retain information, we unroll the recurrence relation:

$$h_t = f(W_h h_{t-1} + W_x x_t + b). \quad (3)$$

Expanding recursively,

$$h_t = f(W_h f(W_h h_{t-2} + W_x x_{t-1} + b) + W_x x_t + b). \quad (4)$$

Continuing for k steps,

$$h_t = f(W_h^k h_{t-k} + \sum_{j=0}^{k-1} W_h^j W_x x_{t-j} + \sum_{j=0}^{k-1} W_h^j b). \quad (5)$$

This shows that RNNs mathematically encode past information into hidden states.

Conditions for Effective Memory Capture

For an RNN to retain memory efficiently, the following must hold:

Eigenvalues of W_h Must Be Near 1

- If $|\lambda_i| > 1$, hidden states grow exponentially (**exploding gradients**).
- If $|\lambda_i| < 1$, past information decays exponentially (**vanishing gradients**).
- If $|\lambda_i| \approx 1$, memory persists for longer sequences.

Recurrent Weight Structure

- If W_h is **diagonalizable** with dominant eigenvalues near 1, long-term memory is better retained.
- If W_h is **sparse** or **low-rank**, memory retention is limited.

Activation Function Choice

- If $f(\cdot)$ (e.g., tanh, sigmoid) **saturates**, gradients vanish, reducing memory.
- ReLU activations can mitigate some vanishing gradient issues.

Theoretical Justification for Memory Capture

From the recursive expansion:

$$h_t = W_h^t h_0 + \sum_{j=0}^{t-1} W_h^j W_x x_{t-j} + \sum_{j=0}^{t-1} W_h^j b. \quad (6)$$

This shows how past states influence future states, confirming RNNs' ability to store temporal dependencies.

The "Kaboom" Problem: Exploding and Vanishing Gradients

A major challenge in training RNNs is the **gradient explosion and vanishing problem**, known as the **Kaboom problem**.

Vanishing Gradient Issue

When backpropagating through time, gradients take the form:

$$\frac{\partial L}{\partial W_h} = \sum_{t=0}^T \left(\prod_{k=t}^T W_h f'(h_k) \right) \frac{\partial L}{\partial h_t}. \quad (7)$$

If $|\lambda_i| < 1$, then W_h^t decays exponentially, leading to vanishing gradients. This prevents long-term dependencies from being learned.

Exploding Gradient Issue

If $|\lambda_i| > 1$, gradients grow exponentially as t increases, leading to instability in training. This results in sudden weight updates that cause the model to diverge (**Kaboom!**).

Solutions to the Kaboom Problem

To mitigate these issues:

- **Gradient Clipping:** Rescale gradients when they exceed a threshold.
- **Orthogonal Initialization:** Ensure W_h has eigenvalues close to 1 at initialization.
- **LSTMs/GRUs:** Use architectures that explicitly control memory retention and gate information flow.

Key Takeaways

1. RNNs propagate past information using hidden states, allowing memory retention.
2. Memory retention depends on the eigenvalues of W_h , activation functions, and weight structure.
3. The "Kaboom problem" (exploding and vanishing gradients) affects RNN training, requiring specialized solutions like LSTMs, GRUs, or gradient clipping.
4. Theoretical and empirical findings confirm that RNNs effectively model sequential data when trained properly.

Backpropagation Through Time (BPTT)

To train RNNs, we use Backpropagation Through Time (BPTT), which unrolls the network across time steps and computes gradients for each weight. The loss function is typically computed as:

$$L = \sum_{t=1}^T \ell(y_t, \hat{y}_t) \quad (8)$$

where ℓ is the loss function (e.g., cross-entropy for classification, mean squared error for regression), and T is the sequence length. For different tasks, the way the loss is applied depends on the type of output:

- **Binary Classification with RNN:** In this case, the model processes a sequence $\{x_1, x_2, \dots, x_T\}$ and produces a final hidden state h_T that is mapped to a single output:

$$y = \sigma(W_y h_T + b_y) \quad (9)$$

where σ is a sigmoid activation function, and the loss is computed using binary cross-entropy:

$$L = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y})) \quad (10)$$

Sentiment Analysis Example: Given a sequence of words in a sentence, the RNN processes each word and produces a final hidden state that represents the sentiment of the entire sentence, which is then classified as positive or negative.

*

Applications of RNNs RNNs are widely used in various domains, including:

- **Natural Language Processing (NLP):** Machine translation, text generation, speech recognition.
- **Time-Series Forecasting:** Stock price prediction, weather forecasting.
- **Video Analysis:** Action recognition, captioning.

RNN-based Seq2Seq Model Overview

When dealing with a Recurrent Neural Network (RNN) for a translation task, such as in sequence-to-sequence (seq2seq) models commonly used in neural machine translation (e.g., English to French), the input sequence (source language) and its translation (target language) often have different lengths. Here's a detailed explanation of what happens and how the RNN architecture handles this situation. A typical seq2seq model consists of two main components:

1. **Encoder:** An RNN (or its variants like LSTM or GRU) that processes the input sequence (source language) and encodes it into a fixed-size context vector.
2. **Decoder:** Another RNN that generates the output sequence (target language) based on the context vector provided by the encoder.

The key challenge in translation is that the source sentence (input sequence) and target sentence (output sequence) can have different lengths due to linguistic differences (e.g., word order, grammatical structure, or vocabulary richness).

What Happens with Different Sequence Lengths?

Input Sequence Processing (Encoder)

- **Input:** Suppose the source sentence is "The cat sits" (3 words).
- **Tokenization:** Each word is tokenized and converted into a vector representation (e.g., using word embeddings via a lookup table like W in Word2Vec).
- **RNN Processing:** The encoder RNN processes the input sequence one token at a time:
 - At timestep $t = 1$: Input "The" $\rightarrow$ Hidden state $h_1 = f(W_{hh}h_0 + W_{xh}x_1)$, where h_0 is the initial hidden state (often zeros), x_1 is the embedding of "The", and f is the activation function (e.g., tanh for vanilla RNN, or more complex for LSTM/GRU).
 - At timestep $t = 2$: Input "cat" $\rightarrow$ Hidden state $h_2 = f(W_{hh}h_1 + W_{xh}x_2)$.
 - At timestep $t = 3$: Input "sits" $\rightarrow$ Hidden state $h_3 = f(W_{hh}h_2 + W_{xh}x_3)$.
- **Context Vector:** After processing all 3 tokens, the final hidden state h_3 (or a combination of hidden states in some models) serves as the context vector c . This vector encapsulates the semantic content of the entire input sequence in a fixed-size representation, regardless of the input length.

Output Sequence Generation (Decoder)

- **Target:** Suppose the translation is "Le chat s'assied" (3 words in French, but could easily be different, e.g., "El gato se sienta" in Spanish with 4 words).
- **Decoder Initialization:** The decoder RNN starts with the context vector c (e.g., h_3 from the encoder) as its initial hidden state $s_0 = c$.
- **Generation Process:** The decoder generates the target sequence one token at a time:
 - At timestep $t = 1$: Input is typically a start token (e.g., $\langle \text{SOS} \rangle \rightarrow$ "start of sequence") $\rightarrow$ Output $y_1 =$ "Le", hidden state $s_1 = f(W_{ss}s_0 + W_{ys}y_0)$.
 - At timestep $t = 2$: Input $y_1 =$ "Le" $\rightarrow$ Output $y_2 =$ "chat", hidden state $s_2 = f(W_{ss}s_1 + W_{ys}y_1)$.
 - At timestep $t = 3$: Input $y_2 =$ "chat" $\rightarrow$ Output $y_3 =$ "s'assied", hidden state $s_3 = f(W_{ss}s_2 + W_{ys}y_2)$.
 - At timestep $t = 4$: Input $y_3 =$ "s'assied" $\rightarrow$ Output $y_4 =$ " EOS_L " (end of sequence), indicating the translation is complete.
- **Output Length:** The decoder continues generating tokens until it outputs an $\langle \text{EOS} \rangle$ token, which can result in a target sequence of any length, not necessarily matching the source length. In this case, the source has 3 words, and the target has 3 words, but it could just as easily have 4 or 5 words depending on the language pair.

Handling Different Lengths

- **Fixed Context Vector:** The encoder compresses the input sequence of any length into a fixed-size context vector c . This abstraction allows the model to handle variable-length inputs without requiring the source and target lengths to match.
- **Decoder Flexibility:** The decoder operates independently of the input length. It generates tokens autoregressively (using the previous output as the next input) until it decides to stop by outputting $\langle \text{EOS} \rangle$. This flexibility accommodates translations of different lengths.
- **Training:** During training, the model uses teacher forcing, where the ground-truth target sequence ("Le chat s'assied") is fed as input to the decoder, and the loss is computed based on the predicted outputs versus the true outputs. The loss function (typically categorical cross-entropy) is applied over the target sequence length, not the source length.

Loss Function

- **Cross-Entropy Loss:** For each timestep in the target sequence, the loss is:

$$\mathcal{L} = - \sum_{t=1}^{T_{\text{target}}} \log P(y_t | y_{<t}, c)$$

where T_{target} is the length of the target sequence (e.g., 3 for "Le chat s'assied"), and $P(y_t | y_{<t}, c)$ is the probability of the correct word y_t given the previous words and the context vector c .

- **No Length Matching:** The loss depends only on the target sequence length T_{target} , not the source sequence length T_{source} (e.g., 3 for "The cat sits"). This ensures the model can optimize regardless of length disparity.

Computing $P(y_t | y_{<t}, c)$

- **Decoder Output:** At each timestep t , the decoder RNN computes a hidden state s_t based on the previous hidden state s_{t-1} , the previous output y_{t-1} (embedded as a vector), and the context vector c . For a vanilla RNN, this is:

$$s_t = f(W_{\text{ss}}s_{t-1} + W_{\text{ys}}y_{t-1})$$

In practice, c might be incorporated into the initial state $s_0 = c$ or concatenated with the input at each step in more advanced models (e.g., with attention mechanisms). For simplicity, we assume c influences s_t indirectly through s_0 .

- **Score Computation:** The hidden state s_t is passed through a linear layer to produce raw scores (logits) for each word in the target vocabulary V :

$$z_t = W_{\text{out}}s_t + b_{\text{out}}$$

where $W_{\text{out}} \in \mathbb{R}^{|V| \times d}$ (with d being the hidden state dimension) and $b_{\text{out}} \in \mathbb{R}^{|V|}$ are the weight matrix and bias of the output layer, respectively. z_t is a vector of size $|V|$, with each element $z_{t,j}$ representing the score for word j .

- **Softmax Probability:** The probability $P(y_t | y_{<t}, c)$ is computed by applying the softmax function to the scores z_t :

$$P(y_t = w_j | y_{<t}, c) = \frac{\exp(z_{t,j})}{\sum_{k=1}^{|V|} \exp(z_{t,k})}$$

where w_j is the j -th word in the vocabulary, and the denominator normalizes the scores over all words in V .

- **Loss Contribution:** For the true target word y_t (e.g., "chat" at $t = 2$), the loss term is $-\log P(y_t | y_{<t}, c)$, which penalizes the model if the probability of the correct word is low.

Practical Example with Different Lengths

- **Source:** "I am happy" (3 words).
- **Target:** "Je suis très heureux" (4 words in French).
- **Encoder:** Processes "I am happy" $\rightarrow h_3 = c$.
- **Decoder:** Generates:
 - $t = 1$: $\langle \text{SOS} \rangle \rightarrow \text{"Je"}$.
 - $t = 2$: $\text{"Je"} \rightarrow \text{"suis"}$.
 - $t = 3$: $\text{"suis"} \rightarrow \text{"très"}$.
 - $t = 4$: $\text{"très"} \rightarrow \text{"heureux"}$.
 - $t = 5$: $\text{"heureux"} \rightarrow \langle \text{EOS} \rangle$.
- **Result:** The model produces a 4-word translation from a 3-word input, showing how the architecture adapts to differing lengths.

Key Takeaways

1. **Encoder-Decoder Decoupling:** The encoder compresses the variable-length input into a fixed-size context vector, decoupling the source length from the target length.
2. **Autoregressive Decoding:** The decoder generates the output sequence step-by-step, stopping at $\langle \text{EOS} \rangle$, allowing the target length to differ from the source length.
3. **Loss Independence:** The loss function is computed based on the target sequence, ensuring training works regardless of length differences.
4. **Flexibility:** This design makes seq2seq RNNs robust to linguistic variations where translations naturally vary in length.

This adaptability is a fundamental strength of seq2seq models with RNNs for translation tasks, enabling them to handle real-world language pairs effectively.

Conclusion

Recurrent Neural Networks (RNNs) are powerful for modeling sequential data but face challenges related to vanishing gradients and long-term dependencies. While LSTMs and GRUs improve on these issues, recent advances like Transformers have largely replaced traditional RNNs in many applications.